

Python Pandas

Pandas is a Python library used for working with data sets. Pandas is an open-source, library which provides high-performance, easy-to-use data structures and data analysis tools for the Python programming language.

It has functions for analyzing, cleaning, exploring, and manipulating data.

The name "Pandas" has a reference to both "Panel Data", and "Python Data Analysis" and was created by Wes McKinney in 2008.

Use of Pandas

- Pandas allows us to analyze big data and make conclusions based on statistical theories.
- Pandas can clean messy data sets, and make them readable and relevant.
- Relevant data is very important in data science(a branch of computer science where we study how to store, use and analyze data for deriving information from it.)

Common Use Cases

- **Finance:** Analyzing stock prices, trading volumes, and time-series data.
- **Healthcare:** Managing patient records and clinical trial results.
- **Business Intelligence:** Cleaning sales data and generating marketing insights.
- **Machine Learning:** Preprocessing and transforming raw data into a format suitable for training AI models.

Core libraries for Data science.

- ✓ Pandas
- ✓ NumPy
- ✓ SciPy
- ✓ Polars
- ✓ Matplotlib
- ✓ Seaborn
- ✓ Plotly

What Can Pandas Do?

Pandas is a powerful Python library used for data manipulation, analysis, and cleaning. It allows you to process massive datasets for machine learning, business intelligence, and scientific research.

Key Capabilities

Read & Write Data: Import and export data from multiple formats including CSV, Excel, SQL databases, JSON, and HTML.

Data Cleaning: Handle missing or "null" values, drop duplicates, fix incorrect data formats, and rename columns to make datasets usable.

Filter & Sort: Easily extract specific rows and columns based on conditions

Data Aggregation (Group By): Group data into categories to perform math (mean, sum, count) across subsets of your data.

Merge & Join: Combine multiple, fragmented datasets together using common identifiers, much like a SQL database.

Time Series Analysis: Work easily with dates and times to calculate rolling averages, shift dates, and resample data over time.

Plotting: Directly visualize data by hooking into libraries like Matplotlib to create histograms, scatter plots, and line charts

Pandas are also able to delete rows that are not relevant, or contains wrong values, like empty or NULL values. This is called cleaning the data.

Pandas Installation

If you have Python and PIP already installed on a system, then installation of Pandas is very easy.

➤ **pip install pandas**

If this command fails, then use a python distribution that already has Pandas installed like, Anaconda, Spyder etc.

How to use Pandas

Once Pandas is installed, import it in your applications by adding the import keyword:

```
import pandas
```

Now Pandas is imported and ready to use.

Example

Import pandas

```
a = [1, 7, 2]
```

```
myvar = pandas.Series(a)
```

```
print(myvar)
```

Standard alias for Pandas is usually pd alias.

alias: In Python alias are an alternate name for referring to the same thing.

Create an alias with the as keyword while importing:

```
import pandas as pd
```

Now the Pandas package can be referred to as `pd` instead of `pandas`.

Example

```
import pandas as pd

data_set = {

    'fruits': ["Apple", "Orange", "Mango"],

    'passings': [3, 7, 2]

}

mydata = pd.DataFrame(data_set)

print(mydata)
```

Checking Pandas Version

The version string is stored under `__version__` attribute.

Example

```
import pandas as pd

print(pd.__version__)
```

Data Structures in Pandas

Series one-dimensional array-like object that can store any data type such as integers, strings, floats, or even Python objects. It comes with labels (called an index).

DataFrame A DataFrame is a two-dimensional, size-mutable tabular data structure with labeled rows and columns, similar to a spreadsheet or SQL table. Each column in a DataFrame is a Pandas Series, allowing you to work with multiple types of data in one table.

Panel A Panel is a three-dimensional container of data. The term "panel data" is derived from econometrics.

The Panel class is deprecated and has been removed in recent versions of Pandas.

Pandas Series

Key Components of Pandas Series

- **Data** The actual information or values stored in the structure.
- **index** (Optional): Labels for the axis.
- **dtype** (Optional): Data type of the Series.
- **name** (Optional): Name of the Series.
- **copy** (Bool): Copy data if True.

Example

Creating a Pandas Series from a list:

```
import pandas as pd
```

```
a = [1, 7, 2]
```

```
myvar = pd.Series(a)
```

```
print(myvar)
```

Pandas Labels

If nothing else is specified, the values are labeled with their index number. First value has index 0, second value has index 1 etc.

This label can be used to access a specified value.

Example

Return the first value of the Series:

```
print(myvar[0])
```

Create Labels

With the index argument, you can name your own labels.

Example

Create your own labels:

```
import pandas as pd
```

```
a = [1, 7, 2]
```

```
myvar = pd.Series(a, index = ["x", "y", "z"])
```

```
print(myvar)
```

When you have created labels, you can access an item by referring to the label.

Example

Return the value of "y":

```
print(myvar["y"])
```

Key/Value Objects as Series

You can also use a key/value object, like a dictionary, when creating a Series.

Example

Create a simple Pandas Series from a dictionary:

```
import pandas as pd

qmarks = {"day1": 420, "day2": 380, "day3": 390}

myvar = pd.Series(qmarks)

print(myvar)
```

Here the keys of the dictionary become the labels.

To select only some of the items in the dictionary, use the index argument and specify only the items you want to include in the Series.

Example

Create a Series using only data from "day1" and "day2":

```
import pandas as pd

qmarks = {"day1": 420, "day2": 380, "day3": 390}

myvar = pd.Series(qmarks, index = ["day1", "day2"])

print(myvar)
```

Convert pandas series to python List

```
.tolist()
```

Example:

```
import pandas as pd

ps = pd.Series([2, 4, 6, 8, 10])

print(ps)

print(type(ps))

print("Convert Pandas Series to Python list")

print(ps.tolist())

print(type(ps.tolist()))
```

DataFrames

Data sets in Pandas are usually multi-dimensional tables, called DataFrames. A Pandas DataFrame is a 2 dimensional data structure, like a 2 dimensional array, or a table with rows and columns.

Difference between Series and DataFrames

➤ *Series is like a column, a DataFrame is the whole table.*

Example

Create a DataFrame from two Series:

```
import pandas as pd
```

```
data = {  
    "qmarks": [420, 380, 390],  
    "duration": [50, 40, 45]  
}
```

```
myvar = pd.DataFrame(data)
```

```
print(myvar)
```

Output

	qmarks	duration
0	420	50
1	380	40
2	390	45

Locate Row

As we can see, the DataFrame is like a table with rows and columns.

Pandas use the loc attribute to return one or more specified row(s)

Example

Return row 0:

#refer to the row index:

```
print(df.loc[0])
```

Output

	qmarks
0	420

```
duration    50
```

```
Name: 0, dtype: int64
```

Note: This example returns a Pandas Series.

Example

Return row 0 and 1:

#use a list of indexes:

```
print(df.loc[[0, 1]])
```

Output

	qmarks	duration
0	420	50
1	380	40

Note: When using `[]`, the result is a Pandas DataFrame.

Named Indexes

With the `index` argument, you can name your own indexes.

Example

Add a list of names to give each row a name:

```
import pandas as pd
```

```
data = {  
    "qmarks": [420, 380, 390],  
    "duration": [50, 40, 45]  
}
```

```
df = pd.DataFrame(data, index = ["day1", "day2", "day3"])
```

```
print(df)
```

Output

	qmarks	duration
day1	420	50
day2	380	40
day3	390	45

Locate Named Indexes

Use the named index in the loc attribute to return the specified row(s).

Example

Return "day2":

```
print(df.loc["day2"])
```

Output

qmarks 380

duration 40

Name: day2, dtype: int64

Modifying DataFrame Row Labels

With the index attribute you can also modify the row labels of a DataFrame.

Example

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Abc', 'Bcd', 'Pqr', 'Xyz'],
    'Age': [32, 28, 45, 38],
    'Gender': ['Male', 'Female', 'Male', 'Female'],
    'Rating': [3.45, 4.6, 3.9, 2.78]},
    index=['r1', 'r2', 'r3', 'r4'])

print(df)

# Modify the Row labels of the DataFrame

df.index = [100, 200, 300, 400]

print('Output df')
```

Accessing The DataFrame Columns Labels

The Pandas `pd.columns` attribute is used to access the labels of the columns in the DataFrame. You can access and modify these column labels similarly to how we work with row labels.

Example

```
import pandas as pd

df = pd.DataFrame({
    'Name': ['Abc', 'Bcd', 'Pqr', 'Xyz'],
    'Age': [32, 28, 45, 38],
    'Gender': ['Male', 'Female', 'Male', 'Female'],
    'Rating': [3.45, 4.6, 3.9, 2.78]},
    index=['r1', 'r2', 'r3', 'r4'])

# Access the column labels of the DataFrame

result = df.columns

print(result)
```

Modifying the DataFrame Column Labels

Column labels can be modified using the columns attribute.

Example

```
import pandas as pd

df = pd.DataFrame({

    'Name': ['Abc', 'Bcd', 'Pqr', 'Xyz'],

    'Age': [32, 28, 45, 38],

    'Gender': ['Male', 'Female', 'Male', 'Female'],

    'Rating': [3.45, 4.6, 3.9, 2.78]}

    index=['r1', 'r2', 'r3', 'r4'])

print(df)

# Modify the Column labels of the DataFrame

df.columns = ['Col1', 'Col2', 'Col3', 'Col4']

print(df)
```

Pandas DataFrame Slicing

Pandas DataFrame slicing is performed using two main attributes, which are –

`.iloc[]`: For slicing based on position (integer-based indexing).

`.loc[]`: For slicing based on labels (index labels or column labels).

Slicing a DataFrame by Position

The Pandas DataFrame.`iloc[]` attribute used to slice a DataFrame based on the integer position (i.e, integer-based indexing) of rows and columns.

Syntax

DataFrame.iloc[row_start:row_end, column_start:column_end]

Where, `row_start` and `row_end` are indicates the start and end integer-based index values of the DataFrame rows. Similarly, `column_start` and `column_end` are the column index values.

Example

```
import pandas as pd
```

```
df = pd.DataFrame([[ 'a','b'], [ 'c','d'], [ 'e','f'], [ 'g','h']], columns=[ 'col1', 'col2'])
```

```
print(df)
```

```
# Slice rows based on position
```

```
result = df.iloc[1:3, :]
```

```
print(result)
```

Output

```
col1 col2
```

```
0  a  b
```

```
1  c  d
```

```
2  e  f
```

```
3  g  h
```

```
col1 col2
```

```
1  c  d
```

```
2  e  f
```

Slicing a DataFrame by Label

The Pandas `DataFrame.loc[]` attribute used to slice a DataFrame based on the labels of rows and columns.

Syntax

***DataFrame.loc[row_label_start:row_label_end,
column_label_start:column_label_end]***

Where, `row_label_start` and `row_label_end` are indicates the start and end labels of the DataFrame rows. Similarly, `column_label_start` and `column_label_end` are the column labels.

Example

```
import pandas as pd

df = pd.DataFrame([[ 'a','b'], [ 'c','d'], [ 'e','f'], [ 'g','h']], columns=[ 'col1', 'col2'],
index=[ 'r1', 'r2', 'r3', 'r4'])

print(df)

result = df.loc['r1':'r3', 'col1']

print(result)
```

Output

```
col1 col2
```

```
r1  a  b
```

```
r2  c  d
```

```
r3  e  f
```

```
r4  g  h
```

```
r1  a
```

```
r2  c
```

```
r3  e
```

```
Name: col1, dtype: object
```

DataFrame Column Slicing

Similar to the above row slicing, Pandas DataFrame Column slicing can also be done using the `.iloc[]` for position and `.loc[]` for labels.

Example

```
import pandas as pd
```

```
data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}
```

```
df = pd.DataFrame(data)
```

```
col_A = df.iloc[:, 0]
```

```
print(col_A)
```

```
# Slice multiple columns
```

```
cols_AB = df.iloc[:, 0:2]
```

```
print(cols_AB)
```

Output

```
0    1
```

```
1    2
```

```
2    3
```

```
Name: A, dtype: int64
```

Slicing multiple columns A and B using `iloc[]`:

```
A B
```

```
0  1  4
```

```
1  2  5
```

```
2  3  6
```

Example: Column Slicing Using loc[]

```
import pandas as pd

data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}

df = pd.DataFrame(data)

# Slice a single column by label

col_A = df.loc[:, 'A']

print(col_A)

# Slice multiple columns by label

cols_AB = df.loc[:, 'A':'B']

print(cols_AB)
```

Output

```
0    1
```

```
1    2
```

```
2    3
```

```
Name: A, dtype: int64
```

```
A    B
```

0 1 4

1 2 5

2 3 6

Modifying Values After Slicing

After slicing a DataFrame, you can modify the sliced values directly. This can be done by assigning new values to the selected elements.

Example

```
import pandas as pd

df = pd.DataFrame([[ 'a', 'b'], [ 'c', 'd'], [ 'e', 'f'], [ 'g', 'h']],
                  columns=['col1', 'col2'])

print(df)

# Modify a subset of the DataFrame using iloc

df.iloc[1:3, 0] = ['x', 'y']

# Display the modified DataFrame

print(df)
```

Output

```
col1 col2
0    a    b
1    c    d
2    e    f
3    g    h
```

Modified DataFrame:

```
col1 col2
0    a    b
1    x    d
2    y    f
3    g    h
```

Dropping Rows using the drop() method

The pandas DataFrame.drop() method is used to remove a specific row from the pandas DataFrame. It can be used to drop rows by their label or position (integer-based index), and it returns a new DataFrame with the selected rows removed.

Example: Dropping DataFrame Rows by Index Values

```
import pandas as pd

df = pd.DataFrame({'A': [1, 2, 3, 4, 5], 'B': [4, 5, 6, 7, 8]})

print(df)

# Drop the row with index 3

result = df.drop(3)

print(result)
```

Output

	A	B
0	1	4
1	2	5
2	3	6
3	4	7
4	5	8

After dropping the row at index 3:

	A	B
0	1	4
1	2	5

2	3	6
4	5	8

Note: This method will raise a `KeyError` if the specified row label or index is not found in the index of the `DataFrame`. And this error can be suppressed by setting the `errors` parameter from `raise` to `ignore`.

Dropping Multiple Rows by Labels

By providing the list of multiple row labels to the `drop()` method, we can easily remove multiple rows at a time from a `DataFrame`.

Example

```
import pandas as pd

df = pd.DataFrame({'A': [1, 2, 3, 4, 5], 'B': [4, 5, 6, 7, 8],
                  'C': [9, 10, 11, 12, 13]}, index=['r1', 'r2', 'r3', 'r4', 'r5'])

print(df)

# Drop the rows by row-labels

result = df.drop(['r1', 'r3'])

print(result)
```

Output

A	B	C	
r1	1	4	9
r2	2	5	10
r3	3	6	11
r4	4	7	12
r5	5	8	13

After dropping the row:

A	B	C	
r2	2	5	10
r4	4	7	12
r5	5	8	13

Removing Rows Based on a Condition

Rows can be removed based on a conditional expression, meaning that you can use a condition inside a selection brackets `[]` to filter the rows. This method is useful when filtering out rows that meet a specific condition, such as missing values or unwanted entries.

Example

```
import pandas as pd
```

```
df = pd.DataFrame({'A': [1, 2, 3, 4, 5], 'B': [4, 5, 6, 7, 8],  
'C': [9, 10, 11, 12, 0]}, index=['r1', 'r2', 'r3', 'r4', 'r5'])  
  
print(df)  
  
# Dropping rows where column 'C' contains 0  
  
result = df[df["C"] != 0]  
  
print(result)
```

Output

	A	B	C
r1	1	4	90
r2	2	5	0
r3	3	6	11
r4	4	7	12
r5	5	8	13

After dropping the row where 'C' has 0:

	A	B	C
r1	1	4	90
r3	3	6	11
r4	4	7	12

r5 5 8 13

Removing Rows using Index Slicing

This is the another approach of removing or dropping rows is using index slicing.
This technique drops a range of rows based on their index positions.

Example

```
import pandas as pd

df = pd.DataFrame({'A': [1, 2, 3, 4, 5], 'B': [4, 5, 6, 7, 8]})

print(df)

# Drop the row using the index slicing

result = df.drop(df.index[2:4])

print(result)
```

Output

	A	B
0	1	4
1	2	5
2	3	6

3	4	7
---	---	---

4	5	8
---	---	---

After dropping the row at 2 and 3:

A	B
---	---

0	1	4
---	---	---

1	2	5
---	---	---

4	5	8
---	---	---